

GBI CLI & Python

A practical guide to moving, archiving and restoring research data.

COPY keeps your originals.
MOVE verifies before removing eligible sources.
RESTORE brings an archive back to files and folders.

Start with a small copy. Check the paths and result. Then use the same tools for your finished datasets and checkpoints.

User guide for GBI 0.4.7
25 September 2026

Find what you need

A first transfer	3
Choosing execution	3
Route and flag matrix	4
Selecting files	5
Archives and restore	5
Resumable chunks	6
Status, waiting and retry	7
Python SDK	7
Cached usage	9
Troubleshooting	10
Data layout and terms	11

Examples use placeholder paths. Run `gbi data roots` and substitute your own paths before copying a command. The installed `--help` describes the version on your cluster.

GBI is the user-owned command line tool for moving data between the mounted HPC filesystems. It can copy or move files between Lustre, FSS and personal Object Storage exposed through the configured mounts. It also has a direct Prefect route for personal Object Storage migrations and a small Python wrapper for use from a training or analysis job.

GBI does not grant access, change Unix ownership, or replace a filesystem administrator. The command runs with the permissions of the user who starts it. Load the site module before using it:

```
module load gbi
gbi --version
gbi data roots
```

This guide accompanies GBI 0.4.7. Use `gbi --version` to check the module you loaded, and `gbi data copy --help` for its supported options.

The `roots` command prints the configured user-facing paths. Use those aliases in commands instead of guessing an internal shard path.

A first transfer

Use `copy` when the source should remain. Use `move` when each unchanged selected source file should be removed only after its copy has been read back and verified. A packed move also verifies the archive container before removing eligible source entries.

```
gbi data copy /your/lustre/results /your/object/results

gbi data move /your/lustre/checkpoints /your/object/checkpoints
```

For a directory, the directory's contents go into the destination directory. For a single file, GBI writes to the named destination file, or places the file under its original name when the destination already is a directory.

A different existing destination is kept and reported as a collision. An identical destination can be independently checked and reused. GBI does not silently overwrite another file.

Moving out of Lustre or FSS normally removes each unchanged selected source file after its verified copy. Moving out of personal Object Storage keeps the Object Storage originals unless `--delete-source` is explicitly supplied. `--delete-source` is valid only with `move`; a partial archive restore keeps its archive container even when that flag is present.

Every selected regular file is hashed during the source read and independently hashed after the destination is written. A move also rechecks the source identity before unlinking it. A worker exit status or a destination that merely exists is not enough to remove a source. Symlinks are represented without following their targets, and special files are rejected.

Choosing execution

The route is selected after a bounded metadata probe unless an explicit route flag is used.

- Small work can run in the current shell. The site settings normally allow up to 8 GiB of selected data in the foreground, but sites may configure this.

- Larger work, an archive whose decoded size is unknown, or a tree that cannot be scanned within the foreground discovery budget is submitted to Slurm.
- When the command is already inside a Slurm allocation, GBI reuses that allocation automatically.
- `--detach` explicitly submits a separate Slurm job. Add `--wait` when a script must wait for that job.
- `--local` is a guard that requires an existing allocation. It is not needed when automatic allocation reuse already applies.
- `--prefect` selects the identity-bound managed Object Storage route. It is a separate migration submission, not a tar archive and not a way to submit an ordinary filesystem-to-filesystem copy.

A foreground Ctrl-C stops the foreground operation. If the terminal is following a submitted Slurm job, Ctrl-C detaches the display; the job keeps running. Use the printed transfer ID to reconnect.

The automatic thresholds are site configuration, not completion-time promises. The foreground probe is deliberately bounded so a huge or slow mounted tree is moved to an execution node rather than causing an unbounded login-node walk.

Route and flag matrix

This table describes the supported combinations. Every cell is explicit. “Yes” means supported on that route; “No” means the parser or route validation rejects the option. Normal source, destination and collision checks still apply.

Option	Ordinary automatic / Slurm	Existing allocation	Prefect
<code>--include</code>	Yes, recursive filename globs	Yes	Yes, up to 100 plain globs
<code>--exclude</code>	Yes; exclusions win	Yes	No
<code>--pack tar/gzip</code>	Yes	Yes	No
<code>--pack-small</code>	Yes; use with <code>--dry-run</code> to review	Yes	No
<code>--chunk-size SIZE</code>	Yes for regular files and new packs	Yes	No
<code>--delete-source</code>	Move only; required for Object Storage sources	Same	No
<code>--dry-run</code>	Yes; no transfer output is written	Yes	Yes
<code>--detach</code>	Yes; submits Slurm	New job; cannot combine with <code>--local</code>	No
<code>--wait</code>	Yes	Yes	Yes
<code>--local</code>	Requires an allocation	Yes	No
<code>--prefect</code>	Selects this route	Mutually exclusive with <code>--local</code>	This is the route

`--pack` and `--pack-small` are mutually exclusive. `--wait` can be combined with `--detach` and is the normal SDK behavior. The CLI parser rejects options that cannot be combined before it starts a transfer.

The Prefect route sends individual files directly to or from personal Object Storage through the identity-bound broker. The ordinary route copies through the configured Lustre, FSS or Alluxio/Object Storage mounts. Ordinary routing is not limited to personal roots: any mounted path that the Unix account can access may be classified and used. Prefect requires one personal Lustre or FSS path and one personal Object Storage path. Within Prefect, every archive or restore request, including a restore to Lustre, must use matching relative source and destination paths. The one exception is the Prefect archive-from-Lustre operation, whose relative names may differ. Different names are otherwise available only on the ordinary route. Prefect paths must be plain paths without wildcards, and reserved transfer-state prefixes are rejected. The route does not support exclusions, packing, chunking or Object Storage source deletion.

Use `gbi data copy --help` and `gbi data move --help` for the installed parser's current wording. If you need packing, exclusions or chunks, omit `--prefect` and keep those options on the ordinary route. If you need direct individual Object Storage objects, keep `--prefect` and remove the unsupported options; do not approximate either route with a second command.

Selecting files

`--include` and `--exclude` match the basename of each file recursively. They are case-sensitive `fnmatch` patterns. An include list is an OR: a file matching any include is selected. An exclude matching any pattern wins over the include. With no includes, ordinary files are selected by default; exclusions still remove matches. Quote patterns so the shell passes them to GBI unchanged.

A common checkpoint selection is:

```
gbi data move /your/lustre/run /your/object/run.gbi.tar \
  --pack tar --include '*.pt' --include '*.pt.*' \
  --exclude '*.tmp'
```

This includes `model.pt` and `model.pt.ready.json`, at any depth, but excludes matching temporary files. It does not mean “all names containing `.pt`”. For example, `.ptx` does not match `*.pt`.

The same selection applies while restoring an archive or a chunk store. For an archive restore, the patterns match each member basename, recursively, rather than the complete member path. For a chunked file, the pattern is tested against the original file name recorded in its manifest. A selection is observed before payload transfer; files added later are not silently included, and a selected file that changes is retained.

Archives and restore

`--pack tar` creates a portable GBI tar archive. `--pack gzip` creates the same format with gzip compression. The destination name must end exactly in `.gbi.tar` or `.gbi.tar.gz`.

Start with tar for many small files: one sequential archive reduces per-file storage operations. Choose gzip when compression saves enough space to justify its CPU cost. Checkpoints may compress poorly, so gzip is not automatically faster. Large individual checkpoints can also suit the direct Prefect route; it keeps them as individual objects. Compare a representative finished folder before choosing a format for a large collection.

```
gbi data copy /your/lustre/run /your/object/run.gbi.tar --pack tar
gbi data move /your/lustre/run /your/object/run.gbi.tar.gz --pack gzip
```

GBI marks and validates its archive format. A plain, unmarked tar file is copied as an ordinary file. A marked archive is recognized even if it was renamed. Within a larger directory, keep generated `.gbi.tar`, `.gbi.tar.gz` and `.gbi-chunks` names so discovery can recognize containers.

Restore an archive to Lustre or FSS with an ordinary copy or move:

```
gbi data copy /your/object/run.gbi.tar /your/lustre/restored-run
```

The restore creates the archive's relative directory layout below the requested destination. Archives preserve relative paths, empty directories, safe symlinks, ordinary modes, modification times and hardlink relationships. They do not restore ownership, ACLs, extended attributes or privileged mode bits. A hardlink whose other name was excluded is restored as a regular file. Destination metadata must be representable. Existing directory permissions are preserved, and a different existing entry is reported as a collision.

A packed move reads the stored archive back and checks its selected inventory before source removal. New or excluded files are never removed by an unconditional recursive cleanup. A partial restore keeps the archive, and a failed or changed source remains available for inspection.

Archive metadata has a 64 MiB supported limit. This limit applies to the file list and source observation inventory, so millions of small files can exceed it with relatively little payload. GBI rejects the request before opening or replacing an output. Choose smaller source folders or inspect candidates with:

```
gbi data copy /your/lustre/run /your/object/run --pack-small --dry-run
```

A dry run is a read-only qualification, not permission to delete a source. It reports incomplete discovery as a lower bound and does not authorize an archive. `--chunk-size` does not increase the archive metadata limit.

Packing small files

`--pack-small` groups qualifying small-file subdirectories before creating archives. The site defaults are at most 64 KiB per file, at least 40 files per group, and at most 1 GiB of selected payload per generated archive. Sites may change these values. Larger siblings remain loose, and each generated archive is named after its source subdirectory with a `.gbi.tar` suffix. The source directory remains the layout root, so a mixed tree contains loose large files alongside generated archives. Restoring that tree unpacks each generated archive into the corresponding relative directory.

Run `--pack-small --dry-run` first when the tree is large. The report is a bounded qualification and does not create archives or authorize source deletion.

Resumable chunks

For one large regular file, `--chunk-size SIZE` writes a versioned `.gbi-chunks` store at the destination, with verified parts and a final manifest. The size is required; examples include 64MiB and 1GiB. Mutable resume journals and locks are kept in the configured Lustre scratch state directory.

```
gbi data move /your/lustre/checkpoint.bin /your/object/checkpoint.bin \
--chunk-size 64MiB
```

A chunked output has a `.gbi-chunks` directory containing the final manifest, parts and completion marker. If the command is interrupted, repeat the same command. GBI checks ownership, part checksums and the source identity before resuming. A partial store is not restorable until its completion marker is valid. Chunk state is kept on the configured Lustre scratch area, outside the destination; it is not staged on a local temporary disk.

Chunking can be combined with creating a new packed archive, which adds the chunk suffix to that archive output. It cannot be used to restore an already encoded archive or chunk store. It is not accepted by the Prefect route.

Status, waiting and retry

For a submitted job, GBI prints a Slurm job ID. For a foreground or allocation transfer, it prints a timestamped transfer ID. Use either where supported:

```
gbi data status 123456
gbi data status 20260925T120000Z-0123456789ab --watch
```

A Slurm numeric ID identifies one transfer when possible. If an existing allocation contains several active transfers, status refuses to guess and lists their transfer IDs. Select the exact ID and retry the status command. A published transfer ID continues to work after the scratch run directory has been replaced by immutable history. Transfers started by an older module may not contain allocation metadata; use the transfer ID printed in their log. Loading a new module does not alter a running request's saved code snapshot.

`--watch` follows until a terminal transfer summary. Without it, status prints one update. For a Slurm failure without a completed summary, inspect the saved run directory and `slurm.out`; already verified records remain the authority. `scancel JOB_ID` is the normal Slurm cancellation command. Cancellation does not undo files already verified and does not authorize deletion of incomplete sources.

The retry command is only for a lost Prefect submission reply:

```
gbi data retry TRANSFER_ID
```

It resends the unchanged saved request after confirming that the transfer belongs to the current user. It is not a retry of a failed Prefect flow and it is not an ordinary Slurm retry. For an ordinary failed or interrupted transfer, inspect receipts and rerun the original copy or move command with the same selection after confirming the source and destination state.

Python SDK

The Python package is a thin, blocking wrapper around the installed CLI. Load the module before starting Python, including when using a virtual environment. The complete public signatures are:

```
copy(source, destination, *, include=(), exclude=(), pack=None,
      pack_small=False, chunk_size=None, dry_run=False, prefect=False)

move(source, destination, *, include=(), exclude=(), pack=None,
      pack_small=False, chunk_size=None, dry_run=False, prefect=False,
      delete_source=False)
```

source and destination accept strings or `pathlib.Path`. include and exclude accept one string or an iterable of strings. pack is None, tar or gzip; chunk_size is a positive size string such as 64MiB. Boolean arguments must be actual booleans.

Both functions add `--wait`, stream the CLI's progress to the current output, and return `subprocess.CompletedProcess` when the CLI exits successfully. A nonzero CLI exit raises `subprocess.CalledProcessError`; an unavailable executable raises `FileNotFoundError`. The SDK does not parse output, submit a second command, or retry a failed command implicitly.

An ordinary SDK call reuses an existing Slurm allocation. Outside Slurm, GBI may submit larger work to Slurm and the Python call still waits. In a training job, call it after all writers have closed their files, usually once from rank zero:

```
from pathlib import Path
from gbi import data

finished = Path("/your/lustre/finished-run")
archive = Path("/your/object/finished-run.gbi.tar")

data.move(finished, archive, pack="tar", include=["*.pt", "*.pt.*"])
```

The selected checkpoint files are removed only after verification; other files stay in the source directory. To retain all originals, use `data.copy`.

Python with an sbatch job

A normal batch script can load the module, run training, then archive the finished output. The destination and source must be accessible on the worker.

```
#!/usr/bin/env bash
#SBATCH --job-name=train-and-archive
#SBATCH --cpus-per-task=2
#SBATCH --time=02:00:00

set -euo pipefail
module load gbi
python train_and_archive.py
```

```

from pathlib import Path
from gbi import data

# Training has finished and closed all checkpoint writers here.
data.move(
    Path("/your/lustre/run"),
    Path("/your/object/run.gbi.tar.gz"),
    pack="gzip",
    include=["*.pt", "*.pt.*"],
)

```

Python Prefect alternative

Set `prefect=True` when the source and destination meet the personal route rules. The call submits individual objects through the site's local broker or authenticated compute-job HTTPS endpoint. It does not create a tar archive.

```

from gbi import data

data.move(
    "/your/lustre/run",
    "/your/object/run",
    include=["*.pt", "*.pt.*"],
    prefect=True,
)

```

The Prefect call cannot use `pack`, `pack_small`, `chunk_size`, `exclude` or `delete_source`. It is a managed migration submission, so a lost reply is recovered with `gbi data retry TRANSFER_ID`, not by blindly submitting a new copy. Its flow receipts remain the authority for per-file verification.

Objects copied through the ordinary Alluxio/Object Storage route may not have the Prefect metadata required by the direct route. To reuse such legacy objects, use the ordinary route so GBI can perform its full destination readback and collision checks; do not add `--prefect` just because the files are in Object Storage.

Cached usage

`gbi data usage` shows live UID quota when the Lustre client is available and a cached owner-scoped inventory snapshot. The snapshot is labelled with its capture time and age. It is a logical apparent-byte report, not allocated filesystem usage, and the command does not recursively scan Lustre.

```

gbi data usage
gbi data usage results --depth 2 --limit 20

```

The optional path must be below your own Lustre root. `--depth` and `--limit` must be positive. A snapshot may be partial or stale; those states are shown. If no snapshot exists, the live quota can still be displayed. Publication is a separate maintained owner process and is not started by this command.

Troubleshooting

“unsupported” or “cannot be combined”

Check the route matrix and the installed `--help`. The most common cases are using `--prefect` with `--pack`, `--exclude`, `--chunk-size` or `--delete-source`. Remove those options and use the ordinary route, or change the operation to one the Prefect route supports. `--pack` and `--pack-small` also cannot be combined.

The destination already exists

An identical destination may be reused after independent verification. A different file or incompatible metadata is retained and reported. Do not delete or overwrite it to make a retry pass. Choose a new destination or reconcile the existing object first.

The source changed or is not stable

GBI retains a source when its type, size, timestamps, inode identity or checksum no longer matches the observed request. Stop writers, create a new snapshot or retry after the application has finished writing. Do not remove `.gbi` locks or partial output to force progress.

Archive metadata exceeds the limit

The 64 MiB limit concerns archive metadata, not just payload bytes. Use `--pack-small --dry-run`, reduce the selected folder, or pack separate smaller folders. A larger `--chunk-size` does not solve an oversized archive manifest.

Status cannot choose a transfer

A numeric allocation ID can refer to several transfers. Use one of the exact transfer IDs printed when those transfers started. If the transfer is already published, the exact ID remains the lookup key. Older module records without allocation metadata may only be reachable by that exact ID.

A Prefect request is “not confirmed”

The broker reply may have been lost after submission. Keep the printed transfer ID and run `gbi data status TRANSFER_ID`. If no submitted run is found, use `gbi data retry TRANSFER_ID` once. Do not use `retry` to repeat a flow that is already reported as failed; inspect its receipts and follow the owner's `retry` procedure.

A mount wait or worker deadline expires

Read the phase, path, receipt directory and Slurm log from the error. A timeout does not prove that a kernel I/O operation stopped. Confirm the job and worker processes have ended before rerunning. Counts printed during discovery or active copying may be incomplete.

Python raises `CalledProcessError`

The CLI returned a nonzero result. The exception is not an automatic retry signal. Inspect the command's receipts and saved output. Python waits for the same result as the shell command and uses the same route restrictions.

Data layout and terms

Active request state lives under the private `.gbi` directory on the Lustre scratch root. Completed history is published below the user's Object Storage mount under `.gbi/transfers/PARENT_ID/TRANSFER_ID/`. For a Slurm or existing-allocation run, `PARENT_ID` is the native Slurm allocation/job ID; for a foreground run it is the transfer ID itself. Follow the exact history path printed by the command and use the printed transfer ID with `status`. The request stores the code hash and route parameters used for the transfer. A receipt records source and destination identity, checksums, verification events and deletion events.

A **plain transfer** is the normal per-file copy or move. A **GBI archive** is a marked tar or gzip container with a manifest and relative member paths. A **chunk store** is a manifest plus verified parts for one logical regular file. A **source fingerprint** is the metadata identity checked before deletion. **Readback** is reopening the destination and verifying it independently. **Object Storage originals** are the files exposed through the personal Alluxio/Object Storage mount. **Prefect route** means the direct individual object migration submitted through the site's identity-bound broker.

GBI protects sources through per-file verification and source rechecks. It does not promise atomic whole-tree transactions: a large transfer can have verified completed files while discovery or another file is still pending. Receipts and terminal state are required to understand what happened.